

OOP 9.vežbe

Matematički fakultet

Kolekcije u Javi

Đurđa Milošević

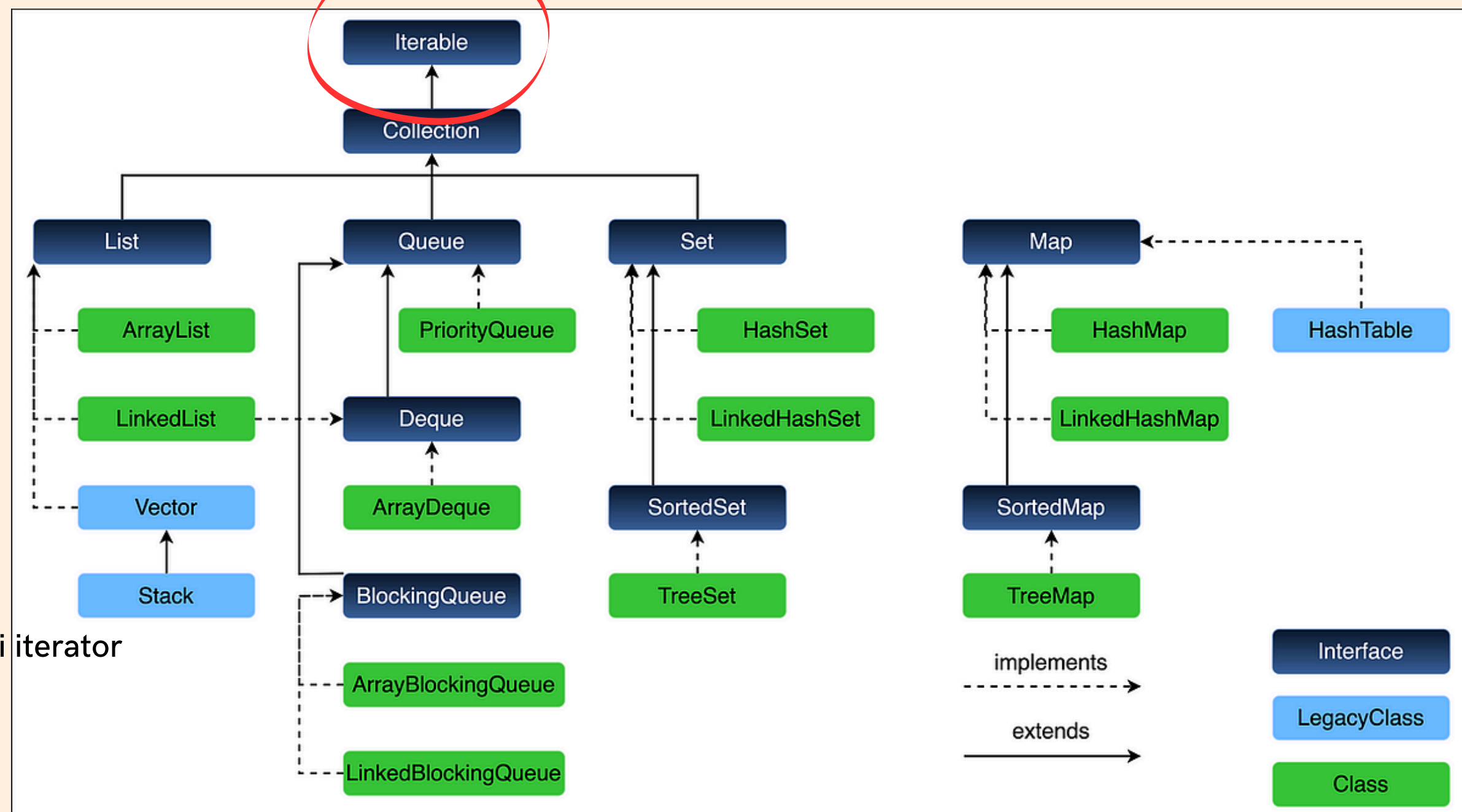
Kolekcije

```
public interface Iterable<T> {  
    Iterator<T> iterator();  
    ...  
}
```

- Java kolekcije organizovane su kroz hijerarhiju interfejsa i klasa
- Najvažnije mogućnosti:
 - dodavanje elemenata
 - uklanjanje elemenata
 - proveru da li kolekcija sadrži element
 - ispitivanje veličine kolekcije
 - **obilazak elemenata**
- Map nije deo Collection hijerarhije

foreach interno koristi iterator

```
for (String s : lista) {  
    System.out.println(s);  
}  
  
public interface Iterator<T> {  
    boolean hasNext();  
    T next();  
    void remove();  
}
```



Implementacije interfejsa Set<T>

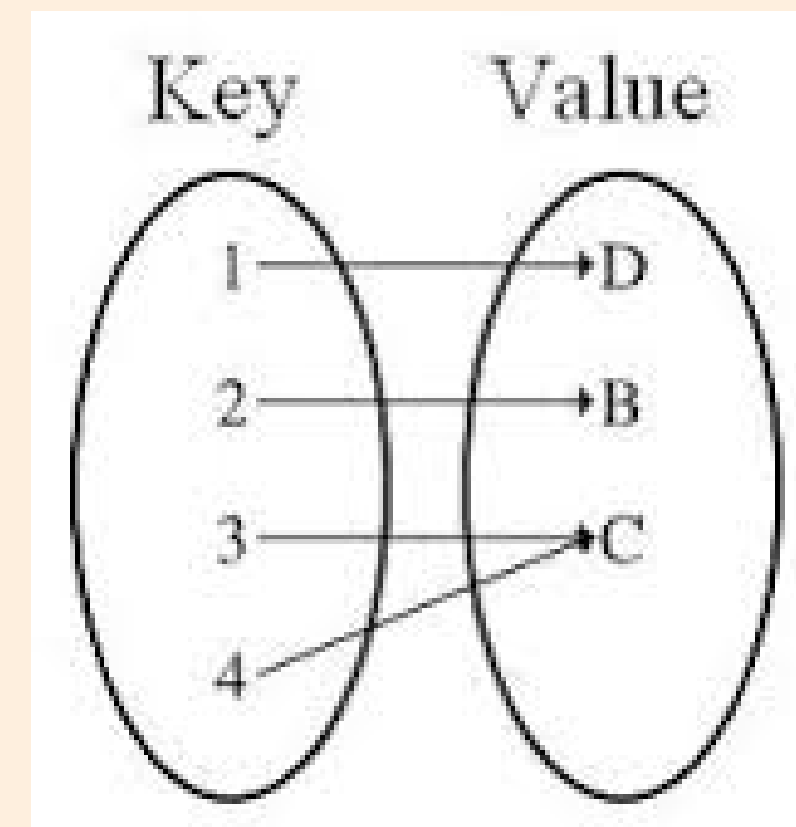
HashSet	TreeSet
elementi nisu sortirani	elementi su sortirani
koristi hash tabelu	koristi balansirano uređeno binarno stablo
veoma brz	sporiji
$O(1)$ prosečno	$O(\log n)$
zahteva hashCode() i equals()	zahteva Comparable ili Comparator

Map<K,V>

- kolekcija koja čuva parove:
 - ključ (K)
 - vrednost (V)
- ključ → vrednost
- ključevi moraju biti **jedinstveni**
- Najčešće implementacije mape su HashMap i TreeMap
- Najvažnije metode:
 - put(K,V)
 - ako pokušamo da dodamo element sa ključem koji već postoji u mapi, stara vrednost će biti zamenjena novom
 - get(K)
 - containsKey(K)
 - remove(K)

```
Map<Integer, String> studenti =  
    new HashMap<>();
```

```
studenti.put(1, "Ana");  
studenti.put(2, "Marko");
```



Tree vs Hash implementacije

TreeSet / TreeMap:

Najvažnije pitanje:

- Kako porediti objekte?

Zbog toga objekti moraju biti uporedivi, odnosno moraju implementirati **Comparable** ili se mora koristiti **Comparator**

HashSet / HashMap:

Najvažnija pitanja:

- Da li su objekti (ključevi) jednaki?
- Gde se objekat nalazi?

Zbog toga, klasa čiji su objekti ključevi mora da override-uje **equals()** i **hashCode()** metode koje su nasleđene iz **Object**-a i imaju podrazumevano ponašanje

Hvala na pažnji!

